CTI modernization plan

Direction and fixed requirements

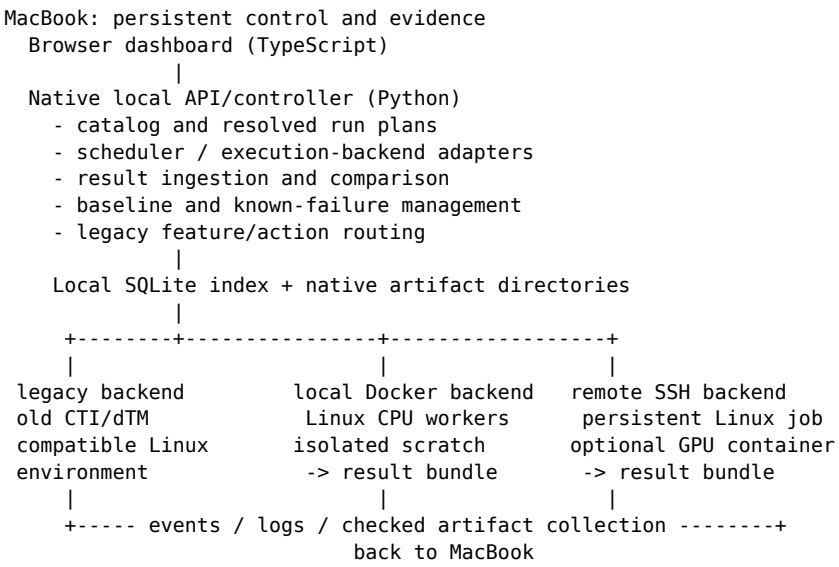
Preserving a usable failed-test debugging environment is the central compatibility requirement of this modernization. Modernize CTI incrementally around a native MacBook dashboard and local evidence store. Retain the existing framework as an executable compatibility path while replacing modules behind explicit contracts. The final harness and dashboard should not require Perl; keeping the old runtime during migration is a temporary compatibility measure, not the target architecture.

This is a proposal, not an implementation. It follows the architecture reconstruction and the seven requirements below. Phase gates describe evidence required before switching a component; they do not claim the current legacy installation has already been validated.

Requirement	Architectural commitment
Preserve CTI failure triage	Retain sources, harness dependencies, compile/ run/ compare scripts, original baseline, workspace evidence, and a replayable container/ mount specification for each failed attempt
Retire Perl	Python for orchestration/ adapters and a TypeScript web UI; limited shell for transparent tool invocation; replace Perl modules by responsibility
Keep old framework operational while swapping modules	Parallel legacy execution path, versioned contracts, per-campaign routing, and reversible cutovers
Preserve dashboard features throughout migration	Feature inventory and acceptance fixtures first; retain working legacy views/ actions until equivalent native workflows pass
Add languages, including the Open64 DSL	Language/ toolchain adapters and phase-based test descriptions, independent of filename extension and execution host
Execute in Docker, keep dashboard and baseline artifacts native to MacBook	Native local service/ storage, isolated Linux workspaces, explicit result collection and baseline versioning
Submit to remote GPU hosts, keep results locally	SSH-based submission, persistent remote job state, resumable artifact transfer, and local ingestion; no shared filesystem requirement

Interpret “native” as a service and files running directly on macOS, with the dashboard viewed in a browser. An AppKit/ Swift desktop application is not required for this initial design. Remote GPU work is assumed to target a supported Linux machine, with its actual GPU vendor/ runtime detected before configuration. The DSL's commands, IR formats, and correctness oracles remain adapter requirements to establish from the Open64 project.

Target structure



Keep one deployable local control service initially, with internal module interfaces rather than separate network services for every responsibility. The legacy backend may execute in a compatible Linux container/ host; the Mac does not need a native Perl CGI installation to browse the new dashboard. dTM remains available for campaigns still using it, and can retire after the replacement backend covers the pools and reservation features actually in use.

The Mac service owns the database and cataloged baseline versions. Workers receive immutable input snapshots, write scratch/ output data, and return result bundles. Workers cannot directly update approved baselines or known-failure annotations.

Language and storage choices

Responsibility	Proposed choice	Reason
CLI, controller, importers, toolchain adapters, workers	Python	Suitable for process/ file orchestration, structured manifests, numerical validation integration, and the existing Python-oriented Open64 work
Dashboard	TypeScript	Typed result/ action contracts and maintainable browser interactions for historical comparisons and large result tables
Small environment/ bootstrap wrappers	Shell where useful	Preserve readable tool commands without moving scheduling, state, or verdict logic into shell
Metadata index	SQLite, accessed only by the native service	Local persistence and query support without running a database server

Responsibility	Proposed choice	Reason
Logs, sources, masters, IR, binaries, measurements	Native content-addressed files plus manifests	Keep large/ binary evidence outside database rows; preserve provenance and original logical filenames
Compatibility runtime	Existing Perl/ shell/ Java/ PHP in a legacy Linux environment	Keep existing features working during component-by-component replacement

Select concrete web/ API frameworks during the first dashboard slice, using feature and table-size tests; the architecture does not depend on an early framework choice. Version public schemas and adapter protocols before introducing framework-specific APIs. Python's subprocess API provides explicit arguments, environments, working directories, timeouts, output capture, and return codes; process-tree cancellation still needs an execution-backend contract.

SQLite remains on the Mac's local disk. Remote workers and containers exchange files/ events with the service, not direct database writes. If WAL mode is selected, keep all database access on that same host; SQLite documents that WAL does not work across network filesystems. See SQLite WAL constraints.

Dashboard continuity is the first delivery track

Establish the compatibility inventory from the actual CGI/ PHP implementation, not just the rewritten guides. Treat viewing, triage, and administrative actions separately so a read-only screen is not mistaken for full parity.

Feature to preserve	Existing implementation evidence	Migration acceptance
Schedule/ options-by-day matrix, totals, setup/ running/ completed/ old-log states	`show-schedule.cgi`, `TestInfo.pm`	Same fixture yields matching counts/ states; options and launcher logs remain accessible
Failure details, categories, filters, include/ exclude buckets, sorting and display limits	`show-failures.cgi`, `chk_test_errors.cgi`	Same test identities and failure classes remain findable; truncation is visible
Previous-run comparisons and newly changed failures	`show-failures.cgi`, `show_regressions.pl`	Matching rules and reference-run choice reproduce characterized examples
Known failures: comments, dates, same-message/ pattern expansion, deletion, import, and text/ grouped reports	Known-failure CGI tools, `bucketize.cgi`, cache helpers	Annotation identity/ meaning survives import; edits do not change raw verdicts

Feature to preserve	Existing implementation evidence	Migration acceptance
Preserved failure workspace and independent compile/ run/ compare replay	Existing driver environment files, generated scripts, source links and retained failure work trees	Open a reconstructed workspace after original container deletion; run each phase against the original baseline; preserve original evidence
Compile/ runtime/ triage logs, generated scripts, options and expected-versus-actual output	`get-*-file.cgi`, `get-build-log.cgi`, `chk_test_errors.cgi`	A failed test can be investigated from the dashboard with original evidence
Rerun preview/ launch, rerun history, test launches from schedules	`rerun_test.cgi`, `start_tests.cgi`	Selected configuration/ test is reproduced and linked as a new attempt
Remaster preview/ application and test enrollment	`remaster-test.cgi`, `addtest.cgi` and helpers	Original operations remain available; native replacement records what changed and who initiated it
Elapsed/ cumulative time, version/ context display, menus and links	Schedule/ failure CGI parameters	Relevant context and navigation are retained
Users, jobs, pools, machine status, enable/ disable, cancellation, performance reservations	`dTMState.php`, `dtmcmd.cgi`, dTM commands	Legacy campaigns retain controls; new jobs expose equivalent supported actions

This is the initial code-derived inventory; phase 0 must enumerate remaining CGI endpoints/ options and turn used behaviors into explicit fixtures. Do not promise pixel-identical rendering or identical defects. Preserve feature semantics and access to original evidence; document intentional bug fixes individually.

The first native dashboard release should view imported legacy results on the Mac. Until each native workflow reaches parity, its legacy view/ action remains reachable through the unified portal. Legacy CGI can stay on a compatible host/ container and be accessed through a narrow compatibility route. This preserves functionality without making the Mac's new dashboard a Perl application. A live legacy environment and representative historical runs must be established to validate this route; the checkout alone is not evidence of a working deployment.

For features not yet replaced, show an explicit legacy action/ link. Do not display a native button whose backend is incomplete. Imported history stays browsable even when workers or compatibility services are offline; a delegated live operation can require its backend to be online.

Contracts that permit incremental replacement

Define a small set of versioned records and backend operations. Prefer module/ process boundaries over translating Perl source line by line.

Contract	Minimum content
Test definition	Stable ID, family, language/ toolchain adapter, source/ input references, phases, expected results, comparators, resource needs
Resolved run plan	Source snapshot, selected tests, compiler build, resolved options with origin, image digest, target architecture, backend, baseline reference
Attempt	Run/ test/ attempt identity, backend job identity, current execution and collection states, timestamps, cancellation information
Phase result	Phase identity, command argument vector, environment reference, exit/ signal/ timeout, stdout/ stderr artifacts, expectation and verdict
Result event	Schema version, attempt ID, monotonically ordered per-attempt event identity, idempotency key, status or artifact manifest
Artifact manifest	Logical path, type, size, digest, provenance and required/ optional collection policy
Failure triage environment	Source/ harness/ baseline snapshots, original and replay scripts, resolved commands/ environment, image/ toolchain references with retention guarantees, workspace evidence, mount graph, target capabilities and preservation/ replay state
Baseline version	Immutable approved oracle/ input/ artifact references, compatibility scope, review record and predecessor
Known-failure annotation	Matching scope, reason/ comment, dates and change history, separate from test outcome and baseline

A language adapter discovers or describes tests and resolves tool-specific phases/ comparators. An execution backend submits, polls, cancels, and collects a resolved plan. A result adapter normalizes legacy or native output. The dashboard reads the same local result/ query API for all three.

Keep raw imported labels alongside normalized statuses, including unknown legacy tags. Preserve the 48 declared result categories and expected-negative-test semantics. Separate execution outcome from collection completeness: a remotely successful test with missing required evidence is not a fully collected successful local result.

Use structured plans and JSON/ JSONL exchange records initially. Tool and schema validation must reject unsupported protocol versions rather than guessing. Retain readable command/

environment reproduction exports, but do not depend on serialized shell environment files as the only run specification.

Coexistence and rollback

Introduce replacement selection per campaign/ unit capability, for example legacy execution with new reporting, native discovery with a validated legacy execution adapter, or a fully native driver on selected units. Do not mix two drivers on the same mutable workspace. Shadow comparisons use separate scratch directories and the same immutable inputs.

For the first slices, import legacy results without changing the old runner. Characterize the log/ parser/ cache behavior before replacing option resolution or discovery. Keep legacy `.opt`, `tmconfig`, and `.sched` usable through the compatibility path. New declarative manifests are additive; converting an existing campaign is a reviewed step.

Legacy `.sched` files are executable Perl and known-failure caches can contain Perl-specific serialized data. Use the compatible legacy reader in an isolated, trusted migration path when needed; do not pretend Python can safely infer those values by parsing source text. Capture normalized exports, raw original files, and import provenance. Any narrowly scoped compatibility glue has an owner and retirement gate; new product modules are not written in Perl.

For each mutable operation, nominate exactly one writer during transition. In particular, known-failure edits, baseline promotions, and schedule edits must not be independently authored in both old and new stores. Before a module switches writers, snapshot its state, drain or queue pending edits, reconcile the import, and verify rollback/ replay. A raw cache restore is not sufficient rollback after new edits have occurred.

The new dashboard can show new-language results without forcing them through an old category model. A compatibility exporter can reproduce legacy logs for the subset that maps faithfully, but does not make old CGI understand arbitrary DSL phases. Users retain old features through the portal while new results use the richer native views.

Docker execution with Mac native evidence

For local Linux testing, use a Linux worker workspace, writable scratch, read-only input/ baseline snapshots, and a dedicated host-backed export directory. The native Mac service verifies and ingests exported manifests, logs, result records, and selected binaries/ IR. Docker Desktop supports sharing Mac paths with its Linux VM, as described in Docker bind mounts.

Do not base the Linux source tree on the existing case-insensitive host checkout. Materialize the Git revision inside a case-sensitive Linux volume, apply an explicitly captured worktree patch when testing uncommitted changes, and record both identities. Represent Mac-native artifacts by digest with original names in manifests so `X.C` and `x.c` can coexist as distinct objects without renaming test identities. Reconstruct original paths in case-sensitive worker storage when rerunning.

Approved baselines stay on the Mac; a container receives an immutable snapshot or requested artifacts and returns proposed replacements. Remastering becomes a reviewed promotion with a preview, preserving the previous baseline. Keep legacy master-file format usable through an export/ materialization adapter where the legacy runner still needs it.

Record native/ container OS, CPU architecture, compiler target, image digest and execution mode separately. If an Apple Silicon Mac runs emulated x86 work, label it and do not compare its timings with native x86 performance baselines. Container execution on the Mac does not imply local access to the remote GPU.

Failed test debugging environments are a core product contract

CTI's retained work environment is its most important value: a developer can inspect the actual failing case, compile it again, execute it, and compare its output with the original baseline. Logs and database rows alone do not satisfy this requirement. Every backend and language adapter must preserve this workflow; dashboard parity includes access to this environment, not merely links to logs.

A failed attempt must have both preserved evidence and a reconstructable writable debugging workspace. The original container need not remain alive, but deleting it must not remove the required state. Preserve the failed workspace before cleanup; do not reconstruct its contents solely by rerunning the test, which could lose the original failure evidence.

Required contents

Retained item	Contract
Test sources and inputs	Exact consumed files, including generated sources, headers, auxiliary files, datasets, uncommitted changes and seeds; stable logical names and digests
Harness dependencies	Unit support objects/ libraries, wrappers, hooks, filters and comparator implementation needed by that test; include shared unit dependencies rather than copying only the leaf source
Compile script	A readable executable entry point preserving compiler invocation, flags, environment, working directory, link steps and required inputs
Execute script	A separate readable entry point preserving arguments, runtime libraries, input redirection, limits and device requirements
Compare script	An independently runnable entry point using the original comparator, baseline, normalization and tolerance settings; it must also accept output from a developer's new run

Retained item	Contract
Baseline	The actual version used by the failed attempt, materialized or retained through a durable local object reference; never silently resolve to the latest master
Failure evidence	Original stdout/ stderr, diagnostic/ output files, phase statuses, generated code/ IR, and relevant objects/ executables; preserve produced core/ debug files where available
Execution environment	Retained image by digest or verifiable image archive, mounted compiler/ runtime/ dependency snapshots, relevant environment and resource/ capability requirements
Mount and path specification	Every required container destination, snapshot identity, access mode, working directory and symlink dependency; separate original host locations from replay locations

A possible materialized debugging view is shown below. Names are proposed interfaces, not files implemented by this documentation change:

```
triage/<run-id>/<test-id>/<attempt-id>/
  manifest.json
  sources/                # materialized from preserved source objects
  inputs/
  baseline/               # baseline used by this attempt
  harness/
  environment/            # resolved settings, image/toolchain, mount graph
  scripts/
    compile.sh
    run.sh
    compare.sh
    replay.sh             # reconstruct container and invoke a phase
    debug-shell.sh        # reopen a writable workspace interactively
  original-workspace/     # evidence preserved at failure
  logs/
```

The native Mac store can physically hold these files in content-addressed storage and materialize this logical view on demand. It must preserve both case-distinct source names even on a case-insensitive Mac filesystem; reconstruction occurs in case-sensitive Linux storage. An archive/ object bundle and a digest manifest are valid native storage, whereas a path to an ephemeral Docker volume alone is not durable local preservation.

Generated shell entry points may call the new Python phase runner. Their contract is that the developer can read the commands and invoke compile, execute and compare independently without needing a live controller, dashboard or database. During legacy execution, retain CTI's original scripts and `.env` files as well as any new launcher wrappers; do not replace those original scripts with only a normalized JSON record.

Mount aware preservation and replay

1. Before execution, capture immutable source/ input/ baseline references, generate the phase scripts, and resolve the required mount graph. Use controlled snapshots for inputs; hashing a mutable host file after a failed run does not establish what the compiler originally read.

2. Mount captured inputs and baseline read-only wherever the adapter permits. Keep mutable build/ run state in a per-attempt workspace. If a legacy suite writes into its source tree, supply a private writable copy seeded from the snapshot, and retain both the initial inputs and failed state.

3. Record original container-visible paths. Replay materializes the saved snapshots and mounts them at the same destinations inside a fresh container, even if the original Mac checkout moved. Do not let preserved scripts follow absolute symlinks into today's mutable checkout or master directory.

4. Freeze and collect the failed workspace, scripts, mount manifest and baseline references before container/ volume cleanup. Capture required files from container layers, volumes and bind mounts; a container image alone does not contain mounted host files or the failed writable workspace.

5. Verify the local bundle and its dependencies, then mark preservation complete. An image digest alone is insufficient if the referenced image can disappear: retain the image or an accessible verified archive and pin external toolchain artifacts needed for replay. Explicitly record prerequisites that must be reprovisioned, rather than claim a self-contained environment when it is incomplete.

6. Open debugging sessions as new writable copies. Compile, run and compare can overwrite their own session outputs but cannot alter the original failed evidence or approved baseline. Record a new attempt/ session relationship when the developer changes a source, option, compiler or baseline.

Use stable container paths for new adapters; legacy adapters may retain their original absolute paths by reconstructing mounts appropriately. Include permissions, executable bits and relevant symlinks. Record needed environment values while replacing credentials with explicit reprovisioning requirements; never silently omit a requirement and call the replay complete.

Shared support harnesses and mounted compilers are dependencies of the debugging environment. Pinning only the test's source revision and image is insufficient when the compiler, runtime, comparator or baseline came from an independently changing mount.

Developer workflow and dashboard behavior

A failed-test page provides access to the preserved sources, original evidence, scripts, baseline and mount/ environment details. It offers Open debugging workspace, Recompile, Execute, Compare and Download triage bundle workflows. These actions use the saved attempt configuration by default. Replaying against a newer compiler or baseline is an explicit new session and retains a link to the original failure.

The debugging workspace supports an interactive shell and, when configured, debugger attachment to a build with suitable symbols and backend capabilities. Interactive debugging is not a promise to checkpoint/ restore the exact live process. For nondeterministic failures, preserving the environment and inputs makes investigation possible but does not guarantee the failure recurs.

For a remote GPU failure, the Mac retains the bundle and can inspect sources, logs and IR offline. Execution/ debug replay requiring the GPU is submitted to a compatible remote target using the saved environment and device requirements. It must not silently fall back to CPU execution or claim that the Mac reproduces the GPU environment.

Retention and acceptance gate

Track preservation separately from the test verdict: pending, complete, incomplete, or failed, with a missing-dependency list. Historical legacy imports may have dangling source links; show that honestly rather than labeling every old work tree replayable. Cancellation, timeout, worker loss and collection errors must not masquerade as fully preserved failures.

Failure preservation overrides normal successful-test cleanup and container auto-removal. Pin its required dependency objects against garbage collection. A controller/ worker should save inputs and scripts before launch and spool failure evidence durably; on disk exhaustion, interruption or host loss, retain what is available, report incompleteness, and retry collection when possible. Remote workers must not remove pending bundles before the Mac acknowledges verified collection. Define an explicit storage-pressure policy instead of silently deleting pinned evidence.

The phase-2 Docker acceptance test must deliberately produce a compile failure, runtime failure and baseline mismatch, preserve each environment, delete the original containers and disposable volumes, move or change the original source checkout and current baseline, and then reconstruct a fresh debugging workspace using only the retained bundle and declared execution prerequisites. Independently recompile, execute where applicable, and compare; verify the original scripts/ evidence/ baseline remain unchanged. Extend this gate to case-distinct filenames, shared unit harnesses and a toolchain supplied by a mount.

Apply the same test to native adapters, the DSL and remote GPU collection before routing production campaigns to them. This is a mandatory exit gate, not a later convenience feature.

Remote GPU execution and return of results

Use an SSH backend first. It should submit a versioned job bundle to a persistent remote job wrapper/ worker, receive a stable remote job ID, and exit without tying the job's lifetime to the SSH connection. Containers are optional per job; GPU requirements belong to the target/ executor contract, not to the DSL parser.

The workflow is: resolve locally, transfer inputs, acknowledge remote job, allocate device resources, run, spool events/ artifacts remotely, reconnect/ poll, copy missing artifacts back, verify digests, ingest locally, and acknowledge collection. Retain remote data until required local collection is acknowledged, subject to an explicit retention policy.

Mac sleep or network loss must not mark a job failed or trigger an automatic duplicate run. Show last-contact time and disconnected/ unknown state. Resumption queries the saved job ID and replays events idempotently. Cancellation is a recorded request until the worker confirms the process/ container stopped. A rerun creates a new attempt rather than overwriting the previous one.

Record device vendor/ model/ UUID, architecture, driver/ runtime/ toolkit, compiler build, image, selected devices, inputs, seed, and measurements. Support exclusive device leases for performance work; testing on a busy shared GPU must not silently produce a comparable performance measurement. Coordinate allocations with all participating jobs on that host; if the host already has a scheduler, integrate with it rather than layering an uncoordinated private lock over it.

For NVIDIA Docker workers, configure the host GPU/ runtime integration and explicit device exposure using the NVIDIA Container Toolkit contract. Other GPU vendors need their own capability/ runtime adapter; NVIDIA is not assumed to be the DSL's only target.

A remote Docker daemon cannot bind-mount the Mac client's files: bind mounts resolve on the daemon's host. Therefore remote jobs need explicit staging and result transfer, even if Docker is also used remotely. See Docker's remote mount constraint.

The local database and collected artifacts remain available after the remote job/ container is removed. Completed history is offline-readable; active monitoring resumes when connectivity returns. Large data and IR retention should be selectable, but required baseline/ result evidence must never be silently omitted.

New language and Open64 DSL testing

Replace extension-to-C/ C++/Fortran assumptions with named toolchain adapters. Keep the test family, language, target, and backend independent: a DSL can have regression, application, and performance tests, each executing locally or remotely as appropriate.

A DSL adapter should describe whatever phases the actual compiler needs, such as parse/ type checking, lowering, IR validation, optimization, code generation, host link, device execution, and numerical comparison. These are candidate extension points, not assumptions about the DSL's implementation.

Support diagnostic expectations, IR snapshots or structural assertions, executable output checks, numerical tolerances/ reference implementations, expected rejection, and measurements. Numerical tolerance, seed/ input generation, device capability, and comparator version must be explicit. Do not reduce DSL testing to adding a suffix that launches a compiler and compares text.

The DSL pilot should use representative real cases from the Open64 project: a positive compiler test, an intentional diagnostic failure, a semantic/ output test, and a remote GPU case if that backend exists. Phase 0 must establish the real compiler commands, dependencies, language identity, oracle, target requirements, and suitable small fixtures before implementing the adapter. The framework should not require a DSL compiler source change just to submit a test.

Delivery sequence and acceptance gates

These phases are ordered by usable deliverables, not estimated calendar dates. Dashboard work starts first and remains an acceptance condition in every later phase.

Phase	Deliverable	Required evidence before advancing
0. Establish compatibility evidence	Failure-triage preservation contract and fixtures, dashboard feature ledger, representative legacy archives/ live campaign, semantic fixtures, schemas, runtime/ target inventory, DSL/ GPU pilot definitions	Archive can be read with the legacy reader; every retained workflow has an owner and acceptance scenario; unavailable historical data is explicitly listed
1. Native dashboard over legacy results	Mac-native result store/ API and useful history/ failure/ artifact browsing; legacy fallback links/ actions	Original failure scripts/ workspaces remain accessible, with incomplete dependencies reported; imported counts, categories, historical comparisons, comments and artifacts reconcile with fixtures; history survives container shutdown; existing operational workflows remain reachable
2. Docker execution vertical slice	Controller submits a small legacy CTI run in Linux Docker and retains failure debugging bundles on Mac	Delete original containers, change/ move original mounts, reopen retained workspace, and independently recompile/ run/ compare against the saved baseline; also pass case-sensitive reconstruction, live status and retry-safe collection
3. Incremental native harness	Python phase runner and regression adapter first; application adapter next; campaign/ unit routing	Native and legacy runs match on characterized subsets including expected failures, timeouts, missing evidence and option overrides; every failed native case passes the retained-environment replay gate; switch back per campaign works
4. Native DSL adapter	Real Open64 DSL tests using the same result/ dashboard contracts	Diagnostic, IR/ semantic and applicable numerical cases preserve sources, phase scripts, comparator/ baseline and failed work state for independent replay; no dashboard specialization required to add a language
5. Remote GPU backend	Persistent SSH submission, capability selection, GPU lease, resumable collection	Mac sleep/ disconnect, worker failure, duplicate event replay, cancellation and collection retry are exercised; locally stored results and failure triage bundles remain after remote cleanup, with GPU replay verified on a compatible target

Phase	Deliverable	Required evidence before advancing
6. Complete native parity and retire legacy modules	Native actions/ reporting/ scheduling and required pool controls; remaining harness migration	All feature-ledger scenarios pass; historical data remains readable; mutations are migrated with rollback; selected campaigns run without Perl or legacy CGI/ dTM dependencies

Schema design in phase 0 must include remote attempt identity and DSL phases so later backends do not force a reporting rewrite. A minimal remote submission experiment can happen early after the contracts stabilize, but dashboard parity and collection durability remain gates. Performance acceptance on real hardware follows successful remote correctness and lease handling.

First implementation scope

Start with the evidence ledger, import contracts, and one dashboard-to-result vertical slice. Use a small legacy regression run and one application result, including a known failure, a new failure, and accessible logs/ masters. Make that history native and durable on the Mac, with a working link to any legacy action not yet replaced.

Then demonstrate a real local Docker run whose output lands in the same dashboard without manual copying and whose failed environment can be reopened after container deletion and original mount changes. That validates the central architecture before replacing compiler-driver logic or expanding scheduling. No existing suite metadata or baseline should be rewritten as a prerequisite.

The completion criterion for the whole transition is concrete: failed tests retain usable source/ compile/ run/ compare debugging environments independently of the original containers and mutable mounts; the Mac dashboard retains the original functional workflows, old and new results coexist, the DSL is supported through adapters, Docker and remote GPU runs collect reliable local evidence, and active campaigns no longer need Perl. Remove compatibility runtimes only when those conditions are demonstrated for the supported campaigns.